Genealogy Web App — Full Build Prompt for Cursor

Paste everything below into Cursor (ideally in Agent/Composer mode, with a fresh empty project folder open). It's written as a single spec so Cursor can scaffold the whole app in stages rather than guessing at architecture.

PROMPT START

You are building a sophisticated, modern genealogy web application called **"Lineage"** (rename as needed). Think of it as **Wikipedia crossed with Ancestry.com**: a collaborative, citation-driven family history platform with a beautiful interactive interface — replacing the dated, PHP-table-driven feel of legacy genealogy sites (e.g. TNG-based sites) with something that feels like a premium modern SaaS product.

Build this in phases, confirming each phase compiles and runs before moving to the next. Use TypeScript everywhere. Do not use `any`. Write clean, componentized, well-commented code.

1. Tech Stack

- **Framework**: Next.js 14+ (App Router, Server Components + Server Actions)
- **Language**: TypeScript (strict mode)
- **Database & Auth**: Supabase (Postgres, Row Level Security, Supabase Auth, Supabase Storage for media)
- **Styling**: Tailwind CSS + shadcn/ui component library
- **Tree visualization**: React Flow (or d3.js) for interactive pedigree/family tree rendering — do NOT use static server-rendered charts
- **Forms**: React Hook Form + Zod validation
- **State/data fetching**: TanStack Query (React Query) for client-side cache, Supabase client for server-side
- **Rich text**: Tiptap editor for biography/notes fields (supports embedded citations and images)
- **Image handling**: Supabase Storage buckets, with client-side compression before upload (browser-image-compression) and automatic thumbnail generation
- **Deployment target**: Vercel

Initialize the project, install all dependencies, and set up the Supabase client (`lib/supabase/client.ts` for browser, `lib/supabase/server.ts` for server components, using `@supabase/ssr`).

2. Core Concept

Two experiences layered on top of one database:

1. **Public/family-facing viewer** — browse the tree, view person profiles, see photos/certificates/timelines, search the archive. Read-only for anonymous visitors (configurable — see privacy rules below).
2. **Wikipedia-style collaborative editor** — authenticated, *authorized* family members can propose or make edits to any person, event, or source record. Every change is versioned. Depending on the user's role, edits either go live immediately or enter a moderation queue.

3. Database Schema (Supabase / Postgres)

Design and create these tables via SQL migrations (`supabase/migrations/`). Use UUID primary keys (`gen_random_uuid()`), `created_at`/`updated_at` timestamps with triggers, and foreign keys with sensible `ON DELETE` behavior.

`profiles` (extends `auth.users`)

- id (uuid, FK to auth.users), full_name, avatar_url, role (enum: `admin`, `editor`, `contributor`, `viewer`), invited_by, bio, created_at

`people`

- id, first_name, middle_names, last_name, maiden_name, nickname, sex (enum: male/female/unknown), birth_date, birth_date_precision (enum: exact/approx/before/after/unknown — genealogy dates are often fuzzy), birth_place_id (FK to `places`), death_date, death_date_precision, death_place_id, is_living (boolean — drives privacy rules), occupation, notes (rich text/JSON from Tiptap), privacy_level (enum: public/family_only/private), created_by, updated_by, current_revision_id

`relationships`

- id, person_id, related_person_id, relationship_type (enum: parent/child/spouse/sibling/guardian), start_date (e.g. marriage date), end_date (e.g. divorce/death), notes

`families` (family units — parents + children, matches gedcom FAM concept)

- id, husband_id, wife_id/partner2_id, marriage_date, marriage_place_id, divorce_date, notes

`family_children` (join table: families ↔ people, with relationship type: biological/adopted/step/foster)

`places`

- id, name, region, country, latitude, longitude, historical_name (places renamed over centuries), parent_place_id (for hierarchical places)

`events` (baptisms, military service, immigration, graduation, etc — extensible beyond birth/death)

- id, person_id, event_type, event_date, event_date_precision, place_id, description, sort_order

`media`

- id, storage_path, media_type (enum: photo/certificate/document/audio/video/headstone), title, description, date_taken, uploaded_by, is_approved

`media_links` (join table: media ↔ people/events/families, so one photo can tag multiple people)

`sources` (citation records — critical for genealogical credibility)

- id, title, source_type (enum: civil_record/church_record/newspaper/census/gravestone/oral_history/dna/other), repository, url, notes

`citations` (join table: sources ↔ any fact — person field, event, relationship)

- id, source_id, cited_table, cited_record_id, cited_field, confidence_level (enum: proven/probable/possible/disproven), page_reference, transcription_excerpt

`revisions` (the Wikipedia engine — every edit to `people`, `events`, `relationships`, `families` is versioned here)

- id, table_name, record_id, editor_id, diff_json (before/after snapshot), edit_summary, status (enum: approved/pending_review/rejected/reverted), reviewed_by, created_at

`talk_comments` (Wikipedia-style "talk page" per person for discussion/disputes)

- id, person_id, author_id, parent_comment_id (for threads), body, created_at

`dna_matches` (optional but valuable — Y-DNA/mtDNA/autosomal notes per person, matches this reference site's DNA storytelling angle)

- id, person_id, haplogroup, test_type, match_notes

`audit_log` — every insert/update/delete across sensitive tables, for accountability.

Enable **Row Level Security** on every table. Write policies so:

- `public`/anon can SELECT people/events/media where `privacy_level = 'public'` and (`is_living = false` OR explicitly marked public)
- `viewer` role can SELECT everything not private
- `contributor` can INSERT/UPDATE but rows land as `pending_review` revisions, not live
- `editor` can INSERT/UPDATE live, bypassing the queue
- `admin` has full control, including role management and merge/delete of duplicate people
- Living people's sensitive fields (birth date, contact info) are hidden from anyone who isn't `family_only` or higher — this is a real genealogy privacy norm, implement it properly

4. Authentication & Authorization

- Supabase Auth with email/password + magic link. Optionally Google OAuth.
- **Invite-only registration**: no public sign-up. Admins generate invite links/codes (store in an `invites` table with expiry) — this mirrors "authorized family members" access control.
- Role-based UI: hide edit/upload controls entirely from `viewer`s; show a "Suggest an edit" flow for `contributor`s that routes to the review queue; show direct edit + "Review pending changes" dashboard for `editor`/`admin`.
- Middleware (`middleware.ts`) to protect `/dashboard` and `/admin` routes server-side, not just client-side.

5. Pages & Routes (App Router)

```
/                          → Landing/homepage: hero, featured ancestor,
recent activity feed, search bar
/tree                      → Interactive full family tree (pan/zoom, React
Flow), click a node to open person panel
/tree/pedigree/[personId]  → Ahnentafel/pedigree chart (ancestors) for a
given person, generational depth selector
/tree/descendants/[personId] → Descendant chart/fan chart
/person/[id]               → Full person profile: photo gallery, biography,
timeline of life events, sourced facts,
```

```
                              family relationships, DNA info, talk/discussion
tab, edit history tab
/family/[id]                → Family group sheet (parents + children),
matches reference site's "family group" pages
/search                     → Advanced search: by name, place, date range,
event type, source
/surnames                   → Alphabetical surname index with counts
/places                     → Places index, ideally with a map view
(Leaflet/Mapbox) plotting events geographically
/media                      → Browsable media gallery, filterable by type
(photo/certificate/document/headstone/etc)
/sources                    → Source/citation library
/timeline                   → Global chronological timeline of all recorded
events, filterable
/whats-new                  → Recent changes feed (Wikipedia "Recent Changes"
equivalent) — who added/edited what, when
/dashboard                  → Authenticated contributor dashboard (see
section 6)
/dashboard/review-queue     → Editors/admins: approve/reject pending
contributor edits, side-by-side diff view
/admin                      → User/role management, invite generation, merge
duplicate people tool, audit log
/auth/sign-in, /auth/invite/[token]
```

6. The Dashboard (Add Photos, Certificates, Bios)

This is the sophisticated content-management core. Build `/dashboard` as a proper app-like interface (sidebar nav, not a form dump):

- **My Contributions** — list of everything the current user has added/edited, with status badges (approved/pending/rejected)
- **Add/Edit Person** — multi-step form (Zod-validated): identity → dates & places (with fuzzy-date picker supporting "circa", "before", "after", "unknown") → relationships (parent/spouse/child pickers with typeahead search against existing `people`) → biography (Tiptap rich text editor, supports inline citation insertion) → media
- **Upload Media** — drag-and-drop multi-file uploader to Supabase Storage, with:
  - Auto-generated thumbnails
  - Required fields: media type, title, date taken (fuzzy-date), tag which person/people/event it belongs to (multi-select, searchable)
  - For certificates specifically: a "transcription" text field so the certificate's content is searchable even before OCR, plus optional structured fields (certificate type:

birth/marriage/death/military/immigration, issuing authority, certificate number)
    - Client-side image compression before upload; show upload progress
- **Add Source/Citation** — form to log a source and attach it to any fact on any person (this is what elevates the app above a typical hobby tree — every fact should be traceable)
- **Bulk GEDCOM Import** — parse uploaded `.ged` files (use a library like `gedcom-parser` or write a minimal parser) and map to the schema, with a preview/conflict-resolution step before committing (detect likely-duplicate people by name+date proximity and let the user merge or create new)
- **Export** — GEDCOM export and PDF family-group-sheet export

Every create/update from the dashboard should write both the live table row (if role permits) and a `revisions` entry capturing the diff, so nothing is ever silently overwritten.

7. The Wikipedia-Style Collaborative System

This is the differentiator — implement it as a real workflow, not just an audit log:

- **Every person/family/event page has an "Edit" button and a "View History" tab**, exactly like Wikipedia.
- **History tab** shows a chronological list of revisions with editor name/avatar, timestamp, edit summary, and a "View diff" action that renders an old-vs-new comparison (highlight added text green, removed text red-strikethrough — build a small diff component, or use `diff` npm package for text fields and a custom renderer for structured fields).
- **Revert** button (editor/admin only) to roll back to a prior revision.
- **Talk/Discussion tab** per person for family members to debate disputed facts (e.g. "are we sure this birth year is right?") before someone edits the record — threaded comments, mentions optional.
- **Review Queue** (`/dashboard/review-queue`, editor/admin only): list of `pending_review` revisions with approve/reject/request-changes actions, and a required reason if rejecting.
- **Conflict handling**: if two people edit the same record concurrently, detect via `updated_at` mismatch and prompt the second editor to review before overwriting (optimistic concurrency, not silent last-write-wins).
- **Notifications** (simple in-app notification bell is enough — no need for email infra unless requested): notify a contributor when their edit is approved/rejected, and notify watchers of a person when that person's page changes ("watch this person" toggle, like Wikipedia watchlists).

8. Design Direction

The explicit goal is to feel dramatically more modern and usable than the reference site (kloosterman.be, a TNG/WordPress genealogy site with dense multi-level flyout menus and dated 2010s styling). Design principles:

- Clean, generous whitespace, modern serif or humanist sans-serif pairing for a "heritage but not dusty" feel (e.g. a serif for names/headings, clean sans for body/UI)
- Dark mode support
- The tree view is the centerpiece: smooth pan/zoom, hover previews on nodes (small photo + name + dates), click to expand a person card without leaving the tree
- Person profile pages read like a well-designed biography page — hero photo, timeline visualization of life events, tabs for Overview / Family / Media / Sources / History / Discussion
- Search should be fast and forgiving — fuzzy matching on names (handle spelling variants, common in old records), instant results dropdown from the nav bar
- Mobile-responsive throughout, including the tree view (touch pan/zoom/pinch)
- Use shadcn/ui primitives (Dialog, Command palette for search, Tabs, DataTable for admin views, Sheet for mobile nav) so the UI is consistent and accessible out of the box

9. Build Order (do this in order, verify each step works before continuing)

1. Scaffold Next.js + Tailwind + shadcn/ui, connect Supabase project, set up env vars (`.env.local` with `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`)
2. Write and run all SQL migrations for the schema above, including RLS policies
3. Build auth flow (invite-only sign-up, sign-in, role-aware middleware)
4. Build person CRUD (server actions) + basic person profile page (no tree yet)
5. Build the revisions system and wire every mutation through it
6. Build the interactive tree view (React Flow) driven by `relationships`/`families`
7. Build the dashboard: add/edit person, media upload, source/citation management
8. Build the review queue and history/diff UI
9. Build search, surname/place indexes, and the recent-changes ("what's new") feed
10. Build GEDCOM import/export
11. Polish: dark mode, mobile responsiveness, loading/empty/error states everywhere, seed script with a handful of sample people/families so the tree isn't empty on first run

10. Non-negotiables

- Every mutation must go through the revisions/audit system — no direct silent writes.
- Every table with personal data must have RLS enabled and tested (try querying as `anon` and as each role to confirm privacy rules actually hold).
- Living people's data must default to restricted visibility.
- No `any` types, no unhandled promise rejections, no client-side secrets.
- Write a `README.md` documenting the schema, role model, and how to run migrations locally via the Supabase CLI.

Confirm your understanding of this spec, then begin with step 1 of the build order.

PROMPT END

A few notes before you paste this in (not part of the prompt itself)

- **Supabase setup**: create the Supabase project first at supabase.com, then give Cursor the project URL and anon/service keys as env vars — don't paste secrets into the prompt itself.
- **Scope**: this is a large build. Cursor will do best if you let it work through the "Build Order" section as a checklist across multiple sessions rather than trying to one-shot the whole thing in a single response — feel free to say "continue with step 4" etc.
- **GEDCOM import** is the most fiddly part (old genealogy software exports messy GEDCOM); expect to iterate on the parser once you test with a real file.
- If you want, I can also draft the initial SQL migration file or the React Flow tree-rendering logic in more depth — just say the word.